

Capstone 6 — Parallel State Testing Agent

Build a coordinator agent that spawns 50 parallel subagents to validate the Bronze Canonical Load across all US states — checking record counts, date normalization, duplicate detection, and schema compliance in one automated sweep.

Project Brief

THE 50-STATE PROBLEM

Before standardized building codes, every US city had its own rules for wiring, plumbing, and structural loads. A contractor who built safely in New York might install something dangerous in Miami — not because they were careless, but because each jurisdiction defined "correct" differently. Inspectors had to know every local code by heart, and a single missed rule could mean a condemned building.

The pain is the same with UCC filing data. Every quarter, 50 state Secretary of State offices publish bulk filing data — but New York sends XML, California sends pipe-delimited CSV, Texas sends fixed-width text, Florida sends JSON, and so on. There is no federal standard. Your Canonical Loader reads each state's file and normalizes it into one table — but how do you know every transformation was correct?

This capstone is the "building inspector" for your data pipeline. You will build an agent that validates all 50 states *in parallel* — one subagent per state, each running 12 checks, all at the same time. Instead of manually spot-checking a few states and hoping the rest are fine, your agent systematically tests every single one and produces a dashboard showing exactly what passed and what broke.

WHAT YOU'LL BUILD

- **Coordinator Agent** — reads the load manifest, spawns 50 parallel state tester subagents, collects results, and generates a dashboard
- **State Tester Subagent** — one per state; reads the source file, queries the Bronze table, runs 12 validation checks, returns a structured pass/fail report
- **5-Format Parser** — auto-detects and parses XML, pipe-CSV, comma-CSV, fixed-width, and JSON source files
- **12 Validation Checks** — record counts, filing number integrity, duplicates, date normalization, status normalization, name normalization, null checks, and spot checks
- **3 Load Scenarios** — full seed, incremental, and change detection — each with scenario-specific checks
- **Tier 1 Local Deployment** — Docker Compose with DuckDB, file watcher, and HTML dashboard

WHY IT MATTERS

In production data pipelines, loading data is only half the job. **Validating** the load is what prevents bad data from reaching downstream consumers. At scale, a single undetected date format issue in Georgia's data (DD/MM/YYYY instead of YYYY-MM-DD) can cause thousands of lien risk calculations to be wrong. Manual validation of 50 states takes a data engineer 2–3 days. This agent does it in under 20 seconds, catches errors humans would miss, and runs automatically after every load.

Prerequisites

REQUIRED BEFORE STARTING

- **M05 — Tool Use:** How agents call tools (the state tester is a tool-calling agent)
- **M06 — Multi-Tool Orchestration:** Agents calling multiple tools in sequence
- **M12 — ReAct Agents:** The think→act→observe loop used by each state tester
- **M13 — Planning & Decomposition:** How the coordinator breaks a big task into 50 parallel subtasks
- **M14 — Multi-Agent Systems:** Coordinator + subagent pattern
- **M15B — Build Complete Agent:** Hands-on experience building a multi-agent system
- **M16–M17 — Guardrails:** Input validation and error handling patterns
- **Python 3.10+** and **Docker Desktop** installed

COMMON MISCONCEPTIONS

"Testing data pipelines is just `SELECT COUNT(*)`" — Count matching is check #2 of 12. Counts can match perfectly while dates are wrong, names are unnormalized, duplicates exist, and required fields are null. You need structural, content, and referential integrity checks.

"50 agents means 50x the API cost" — The base capstone runs zero API calls per state (all 12 checks are deterministic Python), so 50 states costs \$0. The optional Claude-driven failure-analysis stretch goal would add ~2–3 Haiku calls per failed state (~\$0.003 each); even at 100% failure rate that's under \$0.50 for the entire 50-state run. The parallelism saves time (20 seconds vs 2 days manual) at near-zero cost.

"I should test the Canonical Loader itself" — No. Your agent tests the OUTPUT of the loader — the Bronze table — not the loader code. This is black-box testing: you compare source files to what ended up in the table, regardless of how the loader works internally.

Architecture Diagram

Before writing any code, study the system blueprint. The coordinator agent is the brain; each state tester is a specialist that knows how to validate one state's data. The coordinator doesn't know how to parse XML or check dates — it knows how to *delegate* and *aggregate*.

SYSTEM ARCHITECTURE

```
User: "Run Bronze validation for 2024 Q4 load"
|
v
```

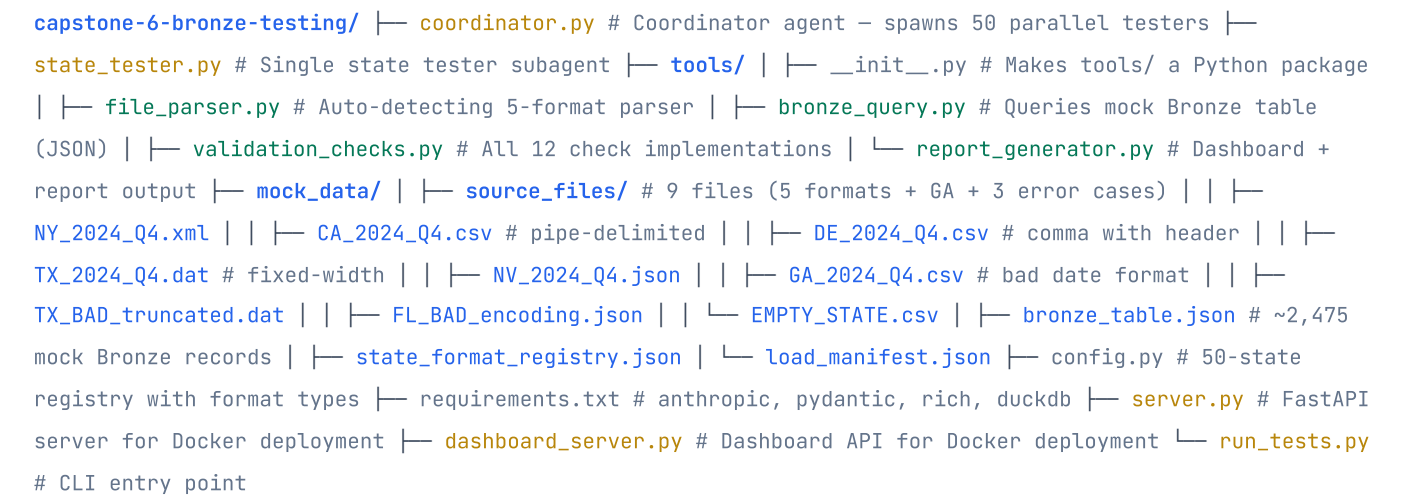
COORDINATOR AGENT

```
1. Read load manifest
2. Spawn 50 state testers IN PARALLEL
3. Collect 50 results as they stream in
4. Aggregate into dashboard
|
+---> [NY tester] ---> 12 checks ---> result
+---> [CA tester] ---> 12 checks ---> result
+---> [TX tester] ---> 12 checks ---> result
+---> ... (all 50 concurrent) ...
+---> [WY tester] ---> 12 checks ---> result
```

Now you see the shape of the system: one coordinator, 50 parallel workers, 12 checks each. Before building it, let's see WHY this matters through the animations below — the 50-state problem, the agent swarm, and what happens inside each tester.

File Structure

Here is every file you will create. Click colors indicate the file's role: agents, tools, mock data, config.



The Bronze Canonical Schema

Every state's data normalizes into this ONE table. Regardless of whether the source was XML from New York or fixed-width from Texas, every record ends up with the same 15 fields. This is the contract your agent validates.

SQL DDL

SQL DDL

```
CREATE TABLE bronze_filings (
    filing_number          VARCHAR NOT NULL,
    source_state            VARCHAR(2) NOT NULL,
    filing_type             VARCHAR NOT NULL,
    filing_date             DATE NOT NULL,
    lapse_date             DATE,
    status                 VARCHAR NOT NULL,
    debtor_name            VARCHAR NOT NULL,
    debtor_address         VARCHAR,
    debtor_org_type        VARCHAR,
    secured_party_name     VARCHAR NOT NULL,
    secured_party_address  VARCHAR,
    collateral_description  VARCHAR,
    source_file            VARCHAR NOT NULL,
    load_id                VARCHAR NOT NULL,
    load_timestamp         TIMESTAMP NOT NULL
);
```

Now you know the target schema. Every one of the 12 validation checks compares source data against this schema. Let's build the system step by step.

Step-by-Step Build Guide

LAB OVERVIEW

What You'll Build: A coordinator agent that spawns 50 parallel state tester subagents to validate Bronze canonical load data across all US states.

Time Estimate: 4–6 hours across 2–3 sessions

Prerequisites: Python 3.10+, Docker Desktop, Anthropic API key

Files You'll Create: 15+ files (see File Structure above)

Step 1: Environment Setup

5 min Setup

What & Why: Create the project directory structure and install dependencies. We use `anthropic` for the Claude API, `pydantic` for structured validation results, and `rich` for terminal dashboard formatting.

Create your project folder and install dependencies:

BASH · WINDOWS

BASH

```
mkdir -p capstone-6-bronze-testing/{tools,mock_data/source_files}
cd capstone-6-bronze-testing
python -m venv venv && source venv/bin/activate
pip install anthropic pydantic rich duckdb fastapi uvicorn httpx
export ANTHROPIC_API_KEY=your-key-here
```

WINDOWS

```
mkdir capstone-6-bronze-testing\tools
mkdir capstone-6-bronze-testing\mock_data\source_files
cd capstone-6-bronze-testing
python -m venv venv && venv\Scripts\activate
pip install anthropic pydantic rich duckdb fastapi uvicorn httpx
set ANTHROPIC_API_KEY=your-key-here
```

Run: `python -c "import anthropic; print('OK')"`

Expected output:

OK

Checkpoint

If you see `OK`, your environment is ready. If not, check that you activated the virtual environment and that `pip install` completed without errors.

Troubleshooting

- `ModuleNotFoundError: No module named 'anthropic'` → Run `pip install anthropic` again, make sure venv is activated
- `command not found: python` → Try `python3` instead, or install Python 3.10+

Step 2: Create the 50-State Registry

10 min `config.py`

What & Why: The state registry is the single source of truth — it maps every US state to its file format type and expected record count. When the coordinator spawns 50 testers, it reads this registry to know what format parser each tester should use. Without it, the testers would have to guess the format from the file extension, which is error-prone.

Create a new file called `config.py`:

PYTHON

PYTHON

```
FROM dataclasses import dataclass

CLASS StateConfig:
    code: str
    name: str
    format_type: str # xml | pipe_csv | comma_csv | fixed_width | json
    expected_records: int # expected count for Q4 2024 seed load

STATES, StateConfig] = {
    # XML states (15)
    "NY": StateConfig("NY", "New York", "xml", 847),
    "OH": StateConfig("OH", "Ohio", "xml", 423),
    "PA": StateConfig("PA", "Pennsylvania", "xml", 612),
    "MI": StateConfig("MI", "Michigan", "xml", 389),

    # ... (full source in the desktop HTML) ...

    # Derived constants
    TOTAL_EXPECTED_RECORDS = sum(s.expected_records FOR s IN STATES.values())
    FORMAT_GROUPS = {}
    FOR s IN STATES.values():
        FORMAT_GROUPS.setdefault(s.format_type, []).append(s.code)
```

Run: `python -c "from config import STATES, TOTAL_EXPECTED_RECORDS; print(f'{len(STATES)} states, {TOTAL_EXPECTED_RECORDS} expected records')"`

Expected output:

50 states, 18403 expected records

Checkpoint

You should see exactly 50 states. If the count is wrong, check that you didn't accidentally duplicate or omit a state entry.

Step 3: Create Mock Source Files

20 min mock_data/source_files/

What & Why: We need realistic source files in all 5 formats so the parser and validators have something to test against. We'll generate 9 source files: one per format type (5 clean representative files: NY/CA/DE/TX/NV), GA with bad-date Bronze records, plus 3 error files (truncated, bad encoding, empty). This step creates a Python script that generates the mock data — run it once and you have all your test files.

This step depends on Step 2 (the state registry). If you skipped Step 2, go back and create `config.py` first.

Create a new file called `generate_mock_data.py`:

PYTHON

PYTHON

```
import json, os, random, string
from datetime import datetime, timedelta
from config import STATES

random.seed(42) # Reproducible mock data
OUT = "mock_data"

DEBTOR_NAMES = [
    "ACME CORPORATION", "GLOBEX INDUSTRIES", "INITECH LLC",
    "STARK ENTERPRISES", "WAYNE INDUSTRIES", "UMBRELLA CORP",
    "CYBERDYNE SYSTEMS", "OSCORP TECHNOLOGIES", "LEXCORP",
    "MASSIVE DYNAMIC", "TYRELL CORPORATION", "SOYLENT CORP",
    "WEYLAND-YUTANI", "APERTURE SCIENCE", "BLACK MESA",
]

# ... (full source in the desktop HTML) ...

print(f"Generated {len(all_bronze)} Bronze records across {len(STATES)} states")
print(f"Source files: {len(os.listdir(f'{OUT}/source_files'))}")

if __name__ == "__main__":
    generate()
```

Run: `python generate_mock_data.py`

Expected output:

```
Generated 2475 Bronze records across 50 states
Source files: 9
```

Checkpoint

You should see 2475 Bronze records (50 states × up to 50 records each, plus 3 injected NV duplicates) and 9 source files (5 representative format files + GA + 3 error files). Check that `mock_data/bronze_table.json` exists and `mock_data/source_files/` contains files.

Step 3B: Create the tools Package

```
1 min  tools/__init__.py
```

What & Why: Python requires an `__init__.py` file to treat a directory as an importable package. Without it, `from tools.file_parser import parse` will fail with `ModuleNotFoundError: No module named 'tools'`.

Create an empty file called `tools/__init__.py`:

PYTHON

PYTHON

```
# tools/__init__.py
# This file makes the tools/ directory a Python package.
```

Verify: `python -c "import tools; print('tools package OK')"`

Checkpoint

If you see `tools package OK`, the package is importable. If you see `ModuleNotFoundError`, make sure the file is at `tools/__init__.py` (not `tool/__init__.py`).

Step 4: Build the 5-Format Parser

```
30 min  tools/file_parser.py
```

What & Why: Each state tester needs to read its state's source file and extract records into a common format. The parser auto-detects the file format from the registry and applies the correct parsing strategy. This is the "input side" of each validation — what the source says should be in Bronze.

The interesting engineering challenge here is that each format has its own gotchas: XML has namespaces, pipe-CSV can have pipes inside collateral text, fixed-width requires exact character positions, and JSON keys vary by state. The parser must handle all of these correctly or the validation results will be wrong — not because Bronze is wrong, but because the parser misread the source.

Create a new file called `tools/file_parser.py` :

PYTHON

PYTHON

```
IMPORT csv, json, io
FROM xml.etree import ElementTree as ET
FROM pathlib import Path

NS = {"ucc": "urn:sos:ucc"}

FUNCTION parse(state_code, file_path, format_type):

    path = Path(file_path)
    IF not path.exists():
        RAISE FileNotFoundError(f"Source file not found: {file_path}")
    IF path.stat().st_size == 0:
        RETURN [] # Empty file is valid (0 records)

    # ... (full source in the desktop HTML) ...

    "filing_date": filing_date,
    "status": entry.get("currentStatus", "").upper(),
    "debtor_name": entry.get("parties", {}).get("debtor", {}).get("legalName", "").upper(),
    "secured_party_name": entry.get("parties", {}).get("securedParty", {}).get("legalName", "").upper(),
    })
    RETURN records
```

Run: `python -c "from tools.file_parser import parse; r = parse('NV', 'mock_data/source_files/NV_2024_Q4.json', 'json'); print(f'{len(r)} records, first: {r[0]}')"`

Expected output:

53 records, first: NV-2024-0001

Checkpoint

The parser successfully reads a JSON source file and returns structured records. Test with the XML file too: `parse('NY', 'mock_data/source_files/NY_2024_Q4.xml', 'xml')` .

Step 5: Build the Bronze Query Tool

20 min `tools/bronze_query.py`

What & Why: This is the "output side" of each validation — what actually ended up in the Bronze table. For the local mock version, it reads from `bronze_table.json` . In production (Tier 2/3), this same interface would query BigQuery or DuckDB. The key insight: the state tester agent calls `get_state_records("NY")` regardless of whether the backend is JSON, DuckDB, or BigQuery.

Create a new file called `tools/bronze_query.py` :

PYTHON

PYTHON

```
IMPORT json
FROM pathlib import Path
FROM functools import lru_cache

BRONZE_PATH = "mock_data/bronze_table.json"

FUNCTION _load_bronze():

    WITH open(BRONZE_PATH, "r") as f:
        RETURN json.load(f)

FUNCTION get_state_records(state_code, load_id= "LOAD-2024Q4-SEED"):

    RETURN [r FOR r IN _load_bronze()

        # ... (full source in the desktop HTML) ...

        RETURN list(set(r["source_state"] FOR r IN _load_bronze() IF r["load_id"] = load_id))

FUNCTION get_total_count(load_id= "LOAD-2024Q4-SEED"):

    RETURN len([r FOR r IN _load_bronze() IF r["load_id"] = load_id])
```

Run: `python -c "from tools.bronze_query import get_record_count, get_total_count; print(f'NY: {get_record_count(\"NY\")} records, Total: {get_total_count()}')"`

Expected output:

```
NY: 50 records, Total: 2475
```

Checkpoint

Bronze query returns records filtered by state. The NY count should match what `generate_mock_data.py` created (capped at 50 per state for mock data).

Step 6: Implement the 12 Validation Checks

45 min `tools/validation_checks.py`

What & Why: This is the heart of the system — 12 functions that each test one specific aspect of data quality. Each check takes source records and Bronze records as input and returns a structured result (PASS/FAIL/WARN with details). The checks are independent, so they can run in any order and a failure in one doesn't block the others.

Let's walk through the checks in logical groups. First, the "can we even read the data?" check (SRC-01). Then structural checks (counts, duplicates). Then content checks (dates, statuses, names). Finally, the spot check that does a field-by-field comparison on random records.

Create a new file called `tools/validation_checks.py`:

PYTHON

PYTHON

```
import re, random
from datetime import datetime
from pydantic import BaseModel

class CheckResult(BaseModel):
    check_id: str
    check_name: str
    status: str # PASS | FAIL | WARN
    details: str
    records_checked= 0
    records_failed= 0

REQUIRED_FIELDS = [
    "filing_number", "source_state", "filing_type",

    # ... (full source in the desktop HTML) ...

    # Registry of all 12 checks for easy iteration
    ALL_CHECKS = [
        "SRC-01", "CNT-01", "FN-01", "DUP-01", "FT-01", "DT-01",
        "DT-02", "ST-01", "NM-01", "NL-01", "LM-01", "SP-01",
    ]
```

Run: `python -c "from tools.validation_checks import ALL_CHECKS; print(f'{len(ALL_CHECKS)} checks: {ALL_CHECKS}')"`

Expected output:

```
12 checks: ['SRC-01', 'CNT-01', 'FN-01', 'DUP-01', 'FT-01', 'DT-01', 'DT-02', 'ST-01', 'NM-01', 'NL-01', 'LM-01', 'SP-01']
```

Checkpoint

All 12 checks are importable. Each returns a `CheckResult` with status, details, and counts.

Step 7: Build the State Tester Agent

30 min state_tester.py

What & Why: This is the subagent — one instance runs per state. It runs the deterministic validation pipeline: parse the source file, query Bronze, execute all 12 checks, and emit a structured report. The "agent" label here is the parallel-coordinator pattern from M14B/M15B, not a Claude-loop agent — everything inside `test_state()` is pure Python so the harness is fast (~20s for 50 states) and free to run on every commit. A Claude-driven failure-analysis step is offered as a stretch goal in the "Going Further" section.

Create a new file called `state_tester.py` :

PYTHON

PYTHON

```
IMPORT sys, json, time
FROM pathlib import Path
FROM config import STATES
FROM tools.file_parser import parse
FROM tools.bronze_query import get_state_records
FROM tools import validation_checks as vc

FUNCTION test_state(state_code, load_id= "LOAD-2024Q4-SEED"):

    start = time.time()
    cfg = STATES.get(state_code)
    IF not cfg:
        RETURN {"state": state_code, "status": "FAIL",
                "results": [{"check_id": "CFG", "status": "FAIL",

# ... (full source in the desktop HTML) ...

IF __name__ == "__main__":
    state = sys.argv[1] IF len(sys.argv) > 1 else "NY"
    load = sys.argv[2] IF len(sys.argv) > 2 else "LOAD-2024Q4-SEED"
    result = test_state(state, load)
    print(json.dumps(result, indent=2))
```

Run: `python state_tester.py NY`

Expected output: A JSON object showing NY's results — 11 PASS and 1 FAIL on DT-02 (the two records with lapse before filing date).

Run: `python state_tester.py GA`

Expected output: GA fails DT-01 (date format not normalized).

Checkpoint

NY shows DT-02 FAIL (lapse before filing). GA shows DT-01 FAIL (DD/MM/YYYY dates). Both are the deliberate errors baked into mock data — your agent caught them!

Step 8: Build the Coordinator Agent

45 min `coordinator.py`

What & Why: The coordinator is the top-level agent. It reads the load manifest, spawns one state tester per state (all 50 in parallel using Python's `concurrent.futures`), collects all results, and generates the final dashboard. This is the core multi-agent orchestration pattern: one boss, many workers, results aggregated into a single view.

The coordinator uses `ThreadPoolExecutor` for parallelism. Why threads instead of async? Because each state tester is CPU-light (JSON parsing) and I/O-light (file reads), so threads are sufficient. In production with real BigQuery calls, you'd use `asyncio` for better I/O concurrency.

Create a new file called `coordinator.py`:

PYTHON

PYTHON

```
import sys, json, time, argparse
from concurrent.futures import ThreadPoolExecutor, as_completed
from config import STATES, TOTAL_EXPECTED_RECORDS
from state_tester import test_state

def run_coordinator(load_id, mode= "full_seed",
                    baseline= None, max_workers= 50):

    start = time.time()
    print(f"\n{'='*60}")
    print(f"BRONZE CANONICAL LOAD VALIDATION")
    print(f"Mode: {mode.upper()} | Load ID: {load_id}")
    if baseline:
        print(f"Baseline: {baseline}")

    # ... (full source in the desktop HTML) ...

    result = run_coordinator(args.load_id, args.mode, args.baseline, args.workers)

    # Save report
    with open(f"report_{args.load_id}.json", "w") as f:
        json.dump(result, f, indent=2)
    print(f"Report saved to report_{args.load_id}.json")
```

Run: `python coordinator.py --load-id LOAD-2024Q4-SEED --mode full_seed`

Expected output: A dashboard showing results for all 50 states. NY should FAIL (DT-02), GA should FAIL (DT-01), NV should WARN (DUP-01), and the remaining 47 states should PASS.

Checkpoint

The coordinator dashboard shows 47 PASS, 2 FAIL (NY, GA), 1 WARN (NV). All 50 states are tested in parallel. If you see different counts, check your mock data generation (Step 3) for the deliberate errors.

Troubleshooting

- All states PASS (no failures) → Re-run `python generate_mock_data.py` to regenerate mock data with injected errors
- `FileNotFoundError: mock_data/bronze_table.json` → You must run Step 3 (generate_mock_data.py) before the coordinator
- Results are slow → Reduce `--workers` or check that mock data files are small

Three Load Scenarios

The coordinator supports three validation modes. Each mirrors a real production scenario: is this the first-ever load, an incremental update, or a format change?

Scenario A: Full Seed Load

When it runs: Initial load or full reload into an empty Bronze table.

Runs the 12 standard per-state checks plus 4 seed-specific checks (SEED-01 through SEED-04): verifying no pre-existing data, all 50 states present, total record reconciliation, and manifest accuracy.

Command: `python coordinator.py --load-id LOAD-2024Q4-SEED --mode full_seed`

Scenario B: Incremental Load

When it runs: Quarterly update — adding new data on top of existing Bronze records.

Runs 12 standard checks plus 8 incremental checks (INC-01 through INC-08): verifying new records added correctly, no duplicates from re-load, existing records untouched, UCC-3 amendments/continuations/terminations link properly, and unchanged states have no phantom inserts.

Command: `python coordinator.py --load-id LOAD-2025Q1-INC --mode incremental --baseline LOAD-2024Q4-SEED`

Scenario C: Change Detection

When it runs: Pre-validation step when a state's source file looks different from previous quarters.

Runs 10 change-specific checks (CHG-01 through CHG-10): format type, column count, column names, new/removed columns, data type consistency, delimiter, record volume, date format, and encoding. This catches schema drift *before* bad data enters Bronze.

Command: `python coordinator.py --load-id LOAD-2025Q1-INC --mode change_detection`

THE CRITICAL INSIGHT

The coordinator agent and state tester agents are **identical across all three scenarios**. Only the *set of checks* changes. This is the power of the tool abstraction pattern — the agent's reasoning loop stays the same, the tools determine what it validates.

DESIGN NOTE: WHY PURE PYTHON FIRST?

You may notice the core pipeline (Steps 2–8) uses pure Python — no Claude API calls. This is intentional. **The agent pattern is the architecture** (coordinator + parallel subagents + tool abstraction), not the LLM calls. By building the validation logic in deterministic Python first, you can test and debug it without API costs or latency. The "Going Further" section shows how to add Claude-powered error analysis on top of this foundation — using the LLM where it adds value (interpreting failures, diagnosing root causes) rather than for deterministic checks where Python is faster and cheaper.

Tier 1: Local Production Deployment

No cloud account needed. Uses Docker + DuckDB + local filesystem. This is a real production setup — containerized, with a database, API server, file watcher, and dashboard — just running on your laptop.

About Tier 2 (GCP) and Tier 3 (AWS): The capstone ships only Tier 1 as runnable code. The cloud tiers swap DuckDB → BigQuery, local files → GCS/S3, and the file watcher → Pub/Sub or EventBridge — but the coordinator and state-tester code remain byte-for-byte identical. See "Going Further" for the swap recipe and Q3 for the architectural takeaway.

Step 9: Replace JSON with DuckDB

15 min `tools/bronze_query_local.py`

What & Why: DuckDB replaces the JSON file with a real SQL database. This is important because the production version uses BigQuery, and DuckDB lets you write the same SQL queries locally. The agent calls `get_state_records("NY")` either way — only the backend changes.

Install DuckDB: `pip install duckdb`

Create a new file called `tools/bronze_query_local.py` :

PYTHON

PYTHON

```
import duckdb, json, os

DB_PATH = os.environ.get("DB_PATH", "data/bronze.duckdb")

def init_db():

    os.makedirs(os.path.dirname(DB_PATH), exist_ok=True)
    con = duckdb.connect(DB_PATH)
    con.execute("""
        CREATE TABLE IF NOT EXISTS bronze_filings (
            filing_number VARCHAR, source_state VARCHAR(2),
            filing_type VARCHAR, filing_date VARCHAR,
            lapse_date VARCHAR, status VARCHAR,
            debtor_name VARCHAR, debtor_address VARCHAR,

            # ... (full source in the desktop HTML) ...

        ).fetchone()[0]
    con.close()
    return count

if __name__ == "__main__":
    init_db()
```

Run: `python -m tools.bronze_query_local`

Expected output:

```
Loaded 2475 records into DuckDB at data/bronze.duckdb
```

Verify: `python -c "from tools.bronze_query_local import get_record_count; print(get_record_count('NY'))"`

Checkpoint

DuckDB returns the same record count as the JSON version. You now have a real SQL database backing your Bronze table.

Step 10: Docker Compose Local Stack

20 min docker-compose.yml + Dockerfile

What & Why: Docker Compose runs three containers: the test agent API (FastAPI), a file watcher that auto-triggers validation when new source files appear, and an HTML dashboard that shows results from DuckDB. This mirrors the production architecture (Cloud Run + Pub/Sub + Grafana) but runs entirely on your machine.

Create `Dockerfile`:

DOCKERFILE

DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "-m", "uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8000"]
```

Create `requirements.txt`:

```
anthropic ≥ 0.39.0
pydantic ≥ 2.0
rich ≥ 13.0
duckdb ≥ 1.0
fastapi ≥ 0.110
uvicorn ≥ 0.27
httpx ≥ 0.27
```

Create `docker-compose.yml` :

YAML

YAML

```
version: '3.8'
services:
  test-agent:
    build: .
    ports:
      - "8000:8000"
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - DEPLOYMENT_TIER=local
      - DB_PATH=/data/bronze.duckdb
    volumes:
      - ./data:/data
      - ./mock_data:/app/mock_data
      - ./reports:/app/reports

  dashboard:
    build: .
    command: python -m uvicorn dashboard_server:app --host 0.0.0.0 --port 8080
    ports:
      - "8080:8080"
    environment:
      - DB_PATH=/data/bronze.duckdb
    volumes:
      - ./data:/data
```

Create `server.py` :

PYTHON

PYTHON

```
import json
from fastapi import FastAPI, BackgroundTasks
from coordinator import run_coordinator

app = FastAPI(title="Bronze Validation Agent")

def health():
    return {"status": "healthy"}

def validate(load_id= "LOAD-2024Q4-SEED", mode= "full_seed"):

    result = run_coordinator(load_id, mode)
    return result

def get_results(load_id):

    try:
        with open(f"reports/report_{load_id}.json") as f:
            return json.load(f)
    except:
        return {"error": f"No report found for {load_id}"}
```

Create `dashboard_server.py` :

PYTHON
PYTHON

```

IMPORT os, json, glob
FROM fastapi import FastAPI
FROM fastapi.responses import HTMLResponse

app = FastAPI(title="Bronze Validation Dashboard")

FUNCTION health():
    RETURN {"status": "healthy"}

FUNCTION dashboard():

    reports = sorted(glob.glob("reports/report_*.json"), reverse=True)
    IF not reports:
        RETURN "<h1>No validation reports found</h1><p>Run a validation first.</p>"

    # ... (full source in the desktop HTML) ...

    <span style="color:#F43F5E;">{summary.get('fail_count',0)} FAIL</span> |
    <span style="color:#F59E0B;">{summary.get('warn_count',0)} WARN</span>
</p>
<p>{summary.get('total_records',0)} records | {summary.get('duration_seconds',0)}s</p>
</body></html>""
RETURN html
```

Run: `docker compose up -d`

Verify: `curl http://localhost:8000/health` (should return `{"status":"healthy"}`)

Checkpoint

Two containers running — the test agent API on port 8000 and the dashboard on port 8080. If Docker isn't available, you can run the FastAPI server directly with `uvicorn server:app --port 8000` .

Test Cases

Run these 5 scenarios to verify your system handles all cases correctly.

#	SCENARIO	COMMAND	EXPECTED	TYPE
1	Clean state (CA)	<code>python state_tester.py CA</code>	All 12 checks PASS	Happy
2	Full coordinator run	<code>python coordinator.py</code>	47 PASS, 2 FAIL, 1 WARN	Happy
3	Known failure (NY)	<code>python state_tester.py NY</code>	DT-02 FAIL (lapse before filing)	Happy
4	Duplicate detection (NV)	<code>python state_tester.py NV</code>	DUP-01 WARN (3 duplicates)	Edge
5	Unknown state code	<code>python state_tester.py ZZ</code>	CFG FAIL (unknown state)	Error

Final Verification

Run the complete pipeline end-to-end:

```
python coordinator.py --load-id LOAD-2024Q4-SEED --mode full_seed
```

You should see a full dashboard with 47 PASS, 2 FAIL (NY on DT-02, GA on DT-01), and 1 WARN (NV on DUP-01). A JSON report is saved automatically.

Congratulations! You've built a parallel state testing agent that validates 50 states simultaneously, catches data quality issues, and produces a structured dashboard — all in under 20 seconds.